MODULE 5: TIME SERIES & PREDICTIVE MODELING (09 PERIODS)

Course Code: 22DS102006 | **B.Tech Computer Science & Engineering (Data Science)**

Detailed 10-Page Comprehensive Study Notes: Time Series Import & Cleaning, Plotting, Moving Averages, Imputation, Autocorrelation, Predictive Modeling & Sentiment Analysis.

1. Introduction to Time Series Analysis

A time series is a sequence of data points indexed in chronological order, typically occurring at uniform time intervals (e.g., hourly stock prices, daily weather readings, monthly sales figures). Time series analysis aims to understand underlying structural patterns, temporal dependencies, and forecast future values.

Unlike standard cross-sectional data where observations are assumed to be independent and identically distributed (i.i.d.), time series data exhibits strong serial correlation, where current values depend heavily on past values.

Core Components of Time Series Decomposition

- **Trend (T_t):** Long-term upward or downward movement in the series.
- **Seasonality (S_t):** Systematic, calendar-related cyclic patterns repeating over fixed periods (e.g., annual or weekly).
- **Residual / Noise (ε_t):** Irregular stochastic fluctuations left over after removing trend and seasonality.

Mathematical Representation

Additive Model: $Y_t = T_t + S_t + \varepsilon_t$. Multiplicative Model: $Y_t = T_t \times S_t \times \varepsilon_t$.

2. Time Series Data Import and Cleaning

Preparing temporal data requires parsing date-time indices, sorting chronological order, resampling frequencies, and handling irregular timestamps or missing intervals.

Key Cleaning Operations in Pandas

- **Datetime Parsing:** Converting string columns into native `DatetimeIndex` objects using `pd.to_datetime()`.
- **Frequency Resampling:** Aggregating high-frequency data into lower intervals (e.g., converting seconds to daily averages via `df.resample('D').mean()`).

```
import pandas as pd import numpy as np # 1. Importing and parsing datetime index raw_data = { 'timestamp': ['2026-01-01 08:00:00', '2026-01-01 10:00:00', '2026-01-03 08:00:00'], 'sensor_val': [22.5, np.nan, 24.1] } df = pd.DataFrame(raw_data) df['timestamp'] = pd.to_datetime(df['timestamp']) df.set_index('timestamp', inplace=True) # 2. Reindexing to fill missing hourly intervals df_regular = df.asfreq('H') print(df_regular.head())
```

> **Chronological Sorting**
>
> Always verify that time series indices are strictly sorted in ascending order (`df.sort_index()`) prior to computing rolling statistics or predictive modeling to prevent severe data leakage.

3. Time Series Visualization & Exploratory Analysis

Visualizing temporal data is the essential first step to identify structural breaks, volatility clustering, non-stationarity, and seasonality.

Plotting Techniques for Temporal Data

- **Line Plots:** The primary plot type showing temporal evolution on the x-axis and metric values on the y-axis.
- **Seasonal Sub-series Plots:** Grouping data by month or day-of-week to inspect intra-period variations across years.
- **Lag Plots:** Scatter plots of Y_t versus Y_{t-1} to visually inspect linear serial correlation.

> **Stationarity Concept**
> A time series is strictly stationary if its statistical properties (mean, variance, autocorrelation) do not change over time. Non-stationary series must be transformed via differencing ($Y'_t = Y_t - Y_{t-1}$) before applying classical forecasting models like ARIMA.

4. Moving Averages and Smoothing Techniques

Moving averages smooth out short-term fluctuations and highlight longer-term trends or cycles in noisy time series data.

Types of Moving Averages

- **Simple Moving Average (SMA):** Unweighted mean of the last k periods: $SMA_t = (1/k) \sum_{i=0}^{k-1} Y_{t-i}$.
- **Weighted Moving Average (WMA):** Assigns linearly decreasing weights to older observations so recent events have higher impact.
- **Exponential Moving Average (EMA):** Applies exponentially decreasing weights, giving recent observations exponentially more weight: $EMA_t = \alpha Y_t + (1 - \alpha)EMA_{t-1}$.

> **Lag Effect of Moving Averages**
> While smoothing noise, larger window sizes (k) introduce a significant time lag, delaying the detection of turning points and trend reversals in predictive applications.

5. Missing Value Imputation in Temporal Data

Missing observations in time series can disrupt rolling window calculations and model training. Standard global imputation methods (like mean imputation) distort temporal variance.

Temporal Imputation Strategies

- **Forward Fill (`ffill`):** Carries the last valid observation forward. Ideal for stock prices or cumulative counters.
- **Linear Interpolation:** Connects missing points with a straight line between the previous and next valid observation.
- **Seasonal Imputation:** Replacing missing values with corresponding values from the same seasonal period in prior cycles.

```
     import pandas as pd import numpy as np # Sample time series with missing values ts =
pd.Series([10.0, np.nan, np.nan, 13.0, 14.0], index=pd.date_range('2026-01-01', periods=5,
freq='D')) # 1. Forward Fill Imputation ts_ffill = ts.ffill() # 2. Linear Interpolation
Imputation ts_interp = ts.interpolate(method='linear') print("Interpolated:\n", ts_interp)
```

6. Serial Correlation & Autocorrelation Functions

Serial correlation measures the relationship between an observation at time Y_t and a lagged observation Y_{t-k}.

Autocorrelation Tools

- **Autocorrelation Function (ACF):** Measures linear correlation between Y_t and Y_{t-k} across all lags k, including indirect intermediary effects.
- **Partial Autocorrelation Function (PACF):** Measures correlation between Y_t and Y_{t-k} while removing the intermediate effects of lags 1 through $k-1$.
- **White Noise:** A series with zero mean, constant variance, and zero autocorrelation across all non-zero lags ($\rho_k = 0$).

> **Role in ARIMA Modeling**
> ACF and PACF plots are the primary diagnostic tools for identifying orders p (autoregressive) and q (moving average) in Box-Jenkins ARIMA modeling.

7. Predictive Modeling & Forecast Evaluation Metrics

Predictive modeling applies supervised machine learning or statistical forecasting algorithms to historical features to predict future outcomes.

Core Evaluation Metrics

- **Mean Absolute Error (MAE):** Average magnitude of absolute errors: $MAE = (1/n) \sum |y_t - \hat{y}_t|$. Robust to outliers.
- **Root Mean Squared Error (RMSE):** Square root of average squared errors: $RMSE = \sqrt{[(1/n) \sum (y_t - \hat{y}_t)^2]}$. Penalizes large errors heavily.
- **Mean Absolute Percentage Error (MAPE):** Percentage-based error metric: $MAPE = (100\%/n) \sum |(y_t - \hat{y}_t) / y_t|$.

> **Time Series Cross-Validation**
> Standard random k-fold cross-validation causes future data leakage in time series. Instead, use **Rolling Origin Cross-Validation (Walk-Forward Validation)** where models are trained exclusively on past data and evaluated on subsequent periods.

8. Building End-to-End Predictive Solutions

Constructing robust predictive pipelines requires feature engineering (lag features, rolling aggregates), model training, and continuous evaluation.

Lag Feature Engineering

Converting time series forecasting into a standard tabular supervised learning problem by generating lag features (Y_{t-1}, Y_{t-2}).

```
      import pandas as pd import numpy as np from sklearn.ensemble import
RandomForestRegressor # Create synthetic time series data df = pd.DataFrame({'sales': [100,
110, 105, 120, 135, 130, 145]}) # Engineer lag features df['lag_1'] = df['sales'].shift(1)
df['lag_2'] = df['sales'].shift(2) df['rolling_mean_3'] =
df['sales'].shift(1).rolling(3).mean() # Drop NaN resulting from shifts df_clean =
df.dropna() print(df_clean.head())
```

> **Feature Engineering Best Practices**
>
> Incorporate calendar attributes (day of week, month, holiday flags) as categorical or cyclical sine/cosine encodings to capture periodic calendar effects.

9. Sentiment Analysis & External Data Integration

Predictive models often benefit from integrating external unstructured text signals, such as customer reviews, news feeds, and social media sentiment scores.

Core NLP Sentiment Pipeline

- **Text Preprocessing:** Lowercasing, tokenization, stop-word removal, and lemmatization.
- **Lexicon-Based Scoring (VADER / TextBlob):** Assigns polarity scores (ranging from -1.0 to +1.0) based on pre-compiled sentiment lexicons without requiring model training.
- **Vectorization (TF-IDF):** Converting text documents into numerical feature matrices: `TF-IDF(t, d) = TF(t, d) × log(N / DF(t))`.

> **Time Series Integration**
>
> Aggregating daily sentiment scores into time-indexed features allows forecasters to combine traditional quantitative metrics (sales history) with qualitative public sentiment indices.

10. Chapter Review & Exam Practice Questions

This final section summarizes core concepts from Module 5 and provides structured practice questions for exam preparation.

```
      # Time Series Evaluation Metric Function import numpy as np def
evaluate_forecast(y_true, y_pred): mae = np.mean(np.abs(y_true - y_pred)) rmse =
np.sqrt(np.mean((y_true - y_pred) ** 2)) mape = np.mean(np.abs((y_true - y_pred) / y_true)) *
100 return {'MAE': mae, 'RMSE': rmse, 'MAPE (%)': mape} actuals = np.array([120, 135, 150,
145]) predictions = np.array([125, 130, 155, 140]) print(evaluate_forecast(actuals,
predictions))
```

Module 5 Exam Review Questions

- Explain why standard random k-fold cross-validation is invalid for time series forecasting and describe walk-forward validation.
- Compare Simple Moving Average (SMA) and Exponential Moving Average (EMA) in terms of responsiveness to recent trends.
- Describe how ACF and PACF plots assist in identifying parameters for temporal forecasting models.